

Competitive Programming

Section 35: Math Mixed Contest

Competitive Programming Course

2026-08-17

Table of contents

1	How to approach a math mixed contest	3
2	Complete local contest set	4
3	A diagnosis routine, not an answer key	6
4	Build formulas from bijections	7
5	Tiny-model validation	8
6	Complexity and arithmetic	9
7	Common mistakes	10
8	Exercises	11

1 How to approach a math mixed contest

Math tasks seldom announce their technique. The task is to turn the story into a precise object before reaching for a formula.

2 Complete local contest set

Problem	Title
A	Blocked Path
B	Coprime Pairs
C	At Least One
D	Complete Alphabet
E	Modular Appointment
F	Festival Synchronization
G	Capped Allocation
H	GCD-One Subsets
I	Exactly K Colors
J	Target LCM Subsets
K	Weighted Coupon Collection
L	Random Divisor Descent
M	Totient Prefix
N	Square-Free Queries
O	GCD Sum
P	Subarray GCD Sum
Q	Coprime Rectangles
R	Exact Divisor Count

Problems F through L form the original harder half of the contest. The added M through R sequence supplies two easy warm-ups, two more difficult problems, and two challenging but bounded problems. These exercises deliberately combine tools instead of isolating one chapter at a time:

Problem	Combination to recognize
F	recognize and apply generalized CRT from Section 30
G	stars and bars plus inclusion-exclusion over violated caps
H	divisor sieve plus subtraction of larger exact gcd classes
I	choose the used colors, then count surjections
J	target factorization plus bitmask union of required exponents
K	tail-sum expectation plus inclusion-exclusion over missing coupons
L	expectation recurrence evaluated by a divisor sieve
M	Euler's totient sieve and prefix sums
N	square divisibility marking and prefix sums

Problem	Combination to recognize
O	classify integers by their exact GCD and use Euler's phi function
P	compress the distinct GCDs of subarrays ending at each position
Q	Möbius inversion plus quotient-interval grouping
R	divisor-count sieve and offline prefix sweep

For these problems, first write the slow or unrestricted mathematical model. The efficient solution usually appears only after identifying repeated divisor work, symmetric bad events, or a small set of prime-factor features.

3 A diagnosis routine, not an answer key

For each local problem, write the following on paper:

What set of objects am I counting, comparing, or assigning probability to?
What makes an object valid?
Can the desired set be partitioned into disjoint cases?
Would it be easier to count its complement or subtract bad objects?
Which assumption makes multiplication or division legal?

Do not infer a method merely from words such as “grid”, “gcd”, or “at least one”. The same words can support different models. The editorial contains the local reductions after you have attempted them.

4 Build formulas from bijections

Every multiplication should have a reason. A product is justified when every object is represented by one independent choice from each factor, with neither omissions nor duplicates. A subtraction is justified when the bad objects form a set whose size you can count exactly. Inclusion-exclusion is appropriate only after you identify the overlapping properties being counted.

For probability, distinguish disjoint events from independent events. Disjoint probabilities add. Independent probabilities multiply. Neither property should be assumed because it seems intuitive.

5 Tiny-model validation

Before coding a closed form, enumerate a tiny instance by hand or with a brute-force checker. Compare three things:

```
the objects in the original statement  
the objects your formula counts  
the boundary cases excluded by the formula
```

This is especially important for empty choices, a value with no factors, impossible states, and modular division.

6 Complexity and arithmetic

Let the largest constraint determine the shape of the solution. If values have a bounded universe, preprocessing may be useful; if a formula contains many queries, precomputation may be necessary. State the cost of every loop and the largest possible intermediate value, not only the final answer.

When working modulo a prime, division means multiplying by an inverse, and only denominators with an inverse are legal. Use `long long` for C++ modular products before reducing. In Python, still reduce deliberately so values and intent remain clear.

7 Common mistakes

Using a product without proving independent choices.
Subtracting overlapping bad sets without correcting double counts.
Treating non-disjoint events as independent.
Forgetting empty or degenerate cases.
Using modular division when the inverse does not exist.

8 Exercises

Attempt A through L without naming a prior technique. For each one, write the mathematical object, one candidate counting identity, and a tiny example that would falsify that identity. Open the editorial only after the identity and its assumptions are explicit.

Run the local checker after the paper trace:

```
CP_TARGET=solution python3 sections/35_math_mixed_contest/check.py
```